The DMM6500 trigger model: a working manual

Ian Ameline · version 1.59 · MIT licence

The trigger model is not a trigger setting. It is a small programmable machine with its own instruction store, running on a second processor, and both the instrument's scanning feature and this application's recording mode are written in it.

Every figure here was measured on this instrument, a DMM6500 on firmware 1.7.17a, unless it cites the manual. Manual citations are by page, to the *DMM6500 Reference Manual*, DMM6500-901-01 Rev. A/April 2018. Where something is still open it says so and says what would settle it. Cross-references name the heading rather than a section number, so they survive a reordering.

---

1. The machine

program store	63 blocks, indexed from 1 (9-48)
sequences	concurrent, each `START` ... `END`; the event log calls them paths
control flow	8 branch types -- always, once, once-excluded, on-event, counter, delta, limit-constant, limit-dynamic -- plus `RESET_BRANCH_COUNT`, which resets a counter and does not branch
signals	8 notify blocks, 2 blenders, 4 timers, 6 digital I/O lines, 8 LAN objects, 3 TSP-Link lines

A program counter, conditional branches, per-block loop counters and inter-sequence signalling. No general storage and no arithmetic, so it is not a computer, but a rendezvous protocol between two sequences is expressible and that is what scanning uses.

The instruction set:

```
BUFFER_CLEAR          [buffer]
MEASURE_DIGITIZE      [buffer [, count]]        count: a value, COUNT_INFINITE,
COUNT_STOP, COUNT_AUTO
WAIT                  event [, clear [, logic, event [, event]]]
DELAY_CONSTANT        seconds
DELAY_DYNAMIC         delayList, index
NOTIFY                trigger.EVENT_NOTIFYN
NOP
LOG_EVENT             eventNumber, message
DIGITAL_IO            bitPattern, bitMask
CONFIG_RECALL         "list" [, index]
CONFIG_NEXT           "list"
CONFIG_PREV           "list"
RESET_BRANCH_COUNT    counter
BRANCH_ALWAYS         block
BRANCH_ONCE           block
BRANCH_ONCE_EXCLUDED  block
BRANCH_ON_EVENT       event, block
BRANCH_COUNTER        targetCount, block
BRANCH_DELTA          targetDifference, block, measureBlock
BRANCH_LIMIT_CONSTANT limitType, limitA, limitB, block [, measureBlock]
BRANCH_LIMIT_DYNAMIC  limitType, limitNumber, block [, measureBlock]
```

Four more exist and are undocumented. `trigger.BLOCK_START`, `trigger.BLOCK_END` and `trigger.BLOCK_CHANNEL_ACTION` are real TSP constants that appear in no `setblock` usage list, and `WRITE_USB` is emitted by `scan.export()`. All four occur zero times in both reference revisions and in the user manual.

The events a `WAIT` or a `BRANCH_ON_EVENT` can name:

```
EVENT_ANALOGTRIGGER   the signal on INPUT HI/LO crossing a comparator level
EVENT_DISPLAY         the front-panel TRIGGER key
EVENT_EXTERNAL        rear EXT TRIG IN
EVENT_COMMAND         a bus trigger (*TRG, GPIB GET, VXI-11 device_trigger)
EVENT_TIMERN          trigger timer N expired            (1..4)
EVENT_NOTIFYN         notify block N executed            (1..8)
EVENT_BLENDERN        blender N fired                    (1..2)
EVENT_DIGION          edge on digital input N            (1..6)
EVENT_LANN            LXI trigger packet on object N     (1..8)
EVENT_TSPLINKN        edge on TSP-Link sync line N       (1..3)
EVENT_SCAN_CHANNEL_READY / EVENT_SCAN_MEASURE_COMPLETE / EVENT_SCAN_COMPLETE /
EVENT_SCAN_ALARM_LIMIT
EVENT_NONE            no event, and illegal as a WAIT's first event
```

Digital I/O, GPIB and TSP-Link need a communications accessory card: KTTI-GPIB, KTTI-TSP or KTTI-RS232.

`trigger.model.state()` returns `EMPTY`, `BUILDING`, `RUNNING`, `WAITING`, `IDLE`, `ABORTED`, `FAILED` or `PAUSED`. `ABORTING` is not observable: 200 samples taken with nothing between them and `abort()` never caught it, so the transition is atomic at that granularity.

The first return value is `userdata`, not a string. It prints as `trigger.STATE_WAITING` and concatenates like one, then raises `-286 string expected, got userdata` the moment anything treats it as one. Compare with `==` against the `trigger.STATE_*` constants. The arity is not fixed either: an `EMPTY` model returned three values, `(state, state, 3)`, and a `FAILED` one printed five, `FAILED FAILED 1 FAILED 0`.

2. Choosing a shape

what you want	the program
sample now, for a fixed count	one `MEASURE_DIGITIZE` block, or `SimpleLoop`
capture on a signal edge and keep what came before it	`MEASURE_DIGITIZE INFINITE` / `WAIT` / `MEASURE_DIGITIZE n`
wait for a device that has not started yet	the same four blocks, with the wait on the comparator
start on an edge or an external marker, whichever comes first	one `WAIT` with `trigger.WAIT_OR` and both events
let a human abort a wait the host cannot interrupt	blend `EVENT_DISPLAY` into the arm's event
record a bursty line and throw the silence away	two measure blocks and a silence detector -- see "The application's recording program"
scan channels	`scan.add()`, and read the program it generates

Four rules cover most of the mistakes.

A `WAIT` on `EVENT_ANALOGTRIGGER` cannot be the first block, because the comparator only evaluates while a measure block runs. A two-block `WAIT` / `MEASURE_DIGITIZE` program on the comparator builds cleanly, sits on the fast path and never fires.

Clear the buffer with `buf.clear()` from Lua before `initiate()` rather than with a `BLOCK_BUFFER_CLEAR`. The block is the one fast-path disqualifier in both canned templates; the Lua call costs nothing.

Use `trigger.CLEAR_ENTER` on any wait a human or a recurring signal can satisfy. The default `CLEAR_NEVER` honours an event that fired before the block was reached.

Read `trigger.model.state()` after every build and every `initiate()`, and drain `eventlog.next()`. Neither call reports its own refusal.

3. Building and running a program

```
trigger.model.load("Empty")                  -- empties the store; any load() replaces
it
trigger.model.setblock(1, trigger.BLOCK_BUFFER_CLEAR, buf)
trigger.model.setblock(2, trigger.BLOCK_WAIT, trigger.EVENT_ANALOGTRIGGER,
                       trigger.CLEAR_ENTER, trigger.WAIT_OR, trigger.EVENT_DISPLAY)
trigger.model.setblock(3, trigger.BLOCK_MEASURE_DIGITIZE, buf, 20000)
trigger.model.initiate()                     -- returns at once; the model runs in
firmware
print(trigger.model.getblocklist())          -- disassembles the store
```

`setblock` does not clear the store. It overwrites the blocks you name and leaves the rest in place, so a 3-block program written over a previous 5-block one leaves blocks 4 and 5 live. A leftover `BRANCH_COUNTER 1000 -> 2` turned a straight-line program into a thousand-iteration loop over the new `WAIT`. Start every build with `load("Empty")`; the template name is case-insensitive.

Any `load()` replaces the store wholesale and truncates it. Against a store of eight `NOP`s:

```
load("SimpleLoop", ...)      5-block template   -> 5 blocks, blocks 6-8 gone
load("LoopUntilEvent", ...)  6-block template   -> 6 blocks, blocks 7-8 gone
load("Empty")                                   -> 0 blocks
trigger.model.abort()                           -> 8 blocks, untouched
trigger.clear()                                 -> 8 blocks, untouched
setblock(1, BLOCK_NOP)                          -> 8 blocks, untouched
```

A short template over a longer program leaves no tail blocks, so a stale tail can only come from `setblock`. `trigger.clear()` clears trigger event detectors rather than the program, which is the opposite of what the name suggests.

`initiate()` is overlapped. It arms the machine and returns; Lua keeps running, and reading the buffer while the model runs works.

The measure function cannot be changed while the model runs, and the refusal is out of band. With a model held in `STATE_WAITING`, `dmm.digitize.func = ...` returned `pcall` true, left the function unchanged, left the state `WAITING`, and posted 2727, "Command prohibited while trigger model is running". Writing the value it already held posted 2727 as well, so the refusal is on the write and not on the value.

Open: whether switching between local and remote control aborts a running model. The manual's local/remote text (15-2) says only that the front panel is disabled and that you are prompted to switch control. One confirmed press settles it: park a model on `EVENT_EXTERNAL` with no marker arriving, poll from inside for 40 s, and touch the screen. `tools/bench_trigpanel.py` case B does that. The press has to be confirmed, because a window in which nobody pressed looks identical to a press the model ignored.

4. Nothing under `trigger.` reports failure

call	`pcall` says	what happened
`setblock(64, ...)`	true	refused -- event 1130, "Parameter block number, expected value from 1 to 63"
`setblock(1, BLOCK_WAIT, EVENT_NONE)` then `initiate()`	true, true	state becomes `STATE_FAILED`, event 2707, "Block number 1 has an invalid event ID in parameter 1"
`dmm.digitize.func = ...` while the model runs	true	discarded -- event 2727, "Command prohibited while trigger model is running"
`trigger.model.pause()` on a model that is not running	true	refused -- event 2767, "Trigger model cannot be paused unless it is already running"
`trigger.model.pause()` on a model that is running	true	honoured: the state becomes `trigger.STATE_PAUSED`, and `resume()` puts it back to `RUNNING`

A block write, an ordinary attribute write and a plain function call each return success and file the reason as an event with its own number. A `pcall` verdict on anything under `trigger.` is therefore a constant, and a program that builds and initiates "successfully" can be dead.

A rejected block reads as a free one, which makes this the first thing to check when a block appears to have no effect. `LOG_EVENT` first measured as free because a probe passed a bare `1` as the event number: `setblock` returned true, the block was never added, and a program of one measure block is on the fast path. The refusal was event 2725, "Log Event block number 2 Event Number is invalid", visible only in the log. With `trigger.LOG_INFO1` (2734) the block is accepted. Confirm a block is present in `getblocklist()` before concluding anything from its absence of effect -- `sdec.gap_model()` in `tsp/serial_core.tsp` reads its whole store back for this reason, counting `MEASURE_DIGITIZE` occurrences as well as checking that each opcode is present.

`setblock(63, ...)` is accepted with no event, so the 63-block ceiling is enforced exactly as documented. The manual's triggering and TSP-command sections describe none of this reporting, and no example in it checks for any of it.

5. Polling a model: what `trigger.model.state()` is worth

The state call is the only verdict the trigger subsystem gives, and it has two failure modes of its own: it lags, and asking for it from the host is not free on a loaded instrument.

The state at `initiate()` is not the state of the wait

`STATE_RUNNING` with nine readings already taken is the model executing the blocks ahead of its wait: a buffer clear, a zero delay and the start of an infinite digitize. Code that reads "not waiting" as "the event has arrived" is wrong on its first look.

Arming takes about 2 ms; admitting it takes another 190

The wait block goes live about 2 ms after `initiate()`, measured two independent ways that agree. From the recorded data, `pre-roll start = fire_s − N5/fs − n_pre/fs` over six captures at the two rates where the post-trigger block dominates the elapsed time gives 1.6 to 2.6 ms. From the block trajectory, `BUFFER_CLEAR` completes at 0.0003 s and the zero `DELAY_CONSTANT` at 0.0012 s. `CLEAR_ENTER` discards any crossing latched before the block was entered, so the crossing that fired is after entry, which means the wait was entered by then.

Time for `trigger.model.state()` to report `STATE_WAITING`, polled from inside the instrument with `delay(0.001)`, 30 arms at each of two capacities:

capacity	min	median	p95	max
21 100	0.1668	0.1917	0.1932	0.2032
860 000	0.0210	0.1928	0.1952	0.1973

The buffer is not a variable. A sweep of seven capacities from 21 100 to 2 800 000 readings, a 133-fold range, is flat at 0.171 to 0.180 s, and five controls are nulls: `dmm.digitize.count` (1 against 817 000), a host `buf.clear()` before `initiate()`, `samplerate` (40 kS/s against 1 MS/s), template `position` (1 against 50), and the poll cadence itself (10 ms, 1 ms and 0.2 ms alike). `buffer.make()` is not inside the interval and is cheap anyway, 2.5 to 3.7 ms at 860 000 readings over 21 observations, running before the model is loaded.

What moves it is what the instrument was doing before the arm, which accounts for the whole spread:

the previous model	time to `STATE_WAITING`
aborted while sitting in `STATE_WAITING`	0.228 s, the worst case
`load("Empty")`, nothing acquiring	0.191 s
a `SimpleLoop` that ran to `IDLE` on its own	0.177 s
a completed `dmm.digitize.read()`	0.164 s

A caller that aborts before every capture always pays the worst row. Aborting for hygiene makes the prior state permanently "aborted while waiting", so the tidiest caller has the least margin. A spin bounded at 0.25 s took its last look at about 0.232 s, 25 iterations at a measured 9.67 ms each, leaving 4 ms of margin at every buffer size; a test of it armed once and then failed three consecutive runs. Use 10 s, forty-four times the slowest arm measured. The distribution has a hard floor at 0.164 s, so trimming the bound buys nothing, and the cost of overshooting it is total: a spin that never sees `STATE_WAITING` has to report an arm that did not go live, because the poll loop cannot tell that from a trigger that already fired, so the recording runs free and captures the silence arming exists to avoid.

Because the 190 ms bounds the firmware's candour rather than its readiness, a capture that fails a short spin was armed the whole time and no data is lost to a late comparator. In twelve latency captures `STATE_WAITING` was never once observed, the signal having fired within one ramp period of the wait going live.

The `delay()` in the spin is load-bearing. Cadence between 10 ms, 1 ms and 0.2 ms is a null, but zero delay is fatal: a tight loop issued 56 102 `pcall(trigger.model.state)` calls over 3 s and the model sat at block 2, the zero `DELAY_CONSTANT`, the whole time, never arming. The state read is IPC to the second processor, and a spin that never yields starves the trigger engine.

Poll from inside the instrument

A wait has to be polled in the same Lua call that initiated it. One chunk that initiates, delays four seconds inside the instrument and then reports:

	state	readings	events
at `initiate()`	`STATE_RUNNING`	9	--
4 s later, nothing sent in between	`STATE_WAITING`	21 100	1, which is 2731 "initiated"
the same program, state polled from the host twice a second	`STATE_RUNNING`, then `STATE_ABORTED` within 0.5 s	frozen at a variable count: 4 500, 9 233, 10 645, 11 756, 14 159 over five runs	2731, then 2728

The abort is filed as event 2728, "Trigger model path #1 has been aborted". Four program shapes behave the same -- `LoopUntilEvent` at position 5 and at position 0, and the hand-built three- and two-block forms -- so it is not the program. Nor is it the block type: a single `print(defbuffer1.n)` from the host ends a model in a measure block, `RUNNING` at n = 2 and `ABORTED` at n = 140, and one parked in a `BLOCK_WAIT` on `EVENT_ANALOGTRIGGER` that cannot fire, with the comparator at 9.9 V against a measured 10 mV idle, `RUNNING` at n = 12 and `ABORTED` at n = 200. Both logs hold 2731 then 2728.

The host's query is not the mechanism. On an instrument minutes out of a power cycle with no application loaded and one socket open, a model parked on `EVENT_EXTERNAL` answered 2440 consecutive host state queries over 4.001 s and was still in `STATE_WAITING` at the last of them.

What ends a model early is a resident application's panel tick, dispatched at 2 Hz whenever the interpreter is idle, which aborts models it did not start. With this application loaded, a model initiated in one chunk with no print in the chunk and no traffic from that socket afterwards still died: three repeats at 427.6, 436.1 and 441.4 ms, holding 17 104, 17 442 and 17 654 readings at 40 kS/s. That is also why the second row above survived four seconds and the third did not. Nothing ends a model whose initiating chunk is still executing, because Lua has not returned and the tick cannot be dispatched. Polled from inside one chunk, the same model reached 39 400 readings.

The first query is answered truthfully, which is what makes this expensive to diagnose. The abort lands behind the reply and the model runs on for about 140 ms, so a host-side probe gets a plausible number back and only the next read shows the damage. One attempt to measure the front-panel key this way returned three arms that all looked like a disabled key; each had been killed by its own readback.

Keithley's `Streaming_Examples/00_Stream_Data_from_DMM6500/Stream_DMM6500.py` queries an initiated model over a raw socket on port 5025, in a loop, for a whole run, and depends on the model continuing: its `get_data()` blocks inside the instrument until the buffer grows, so an abort on the first query would deadlock the host, and it never calls `trigger.model.abort()`. Nothing resident may tidy models away for that to work. The clean-room measurement above covers a parked wait under state queries for 4 s rather than a whole streaming run, so it does not settle that script either way. Of the other two streaming examples, `KEIDMM6500_Stream_Measured_Dual_Meter.py` talks USBTMC through PyVISA rather than a socket and pairs a DMM6500 with a DAQ6510, so it says nothing about this interface.

6. `BLOCK_WAIT` and the latch

```
setblock(n, trigger.BLOCK_WAIT, event [, clear [, logic, event [, event]]])
```

`logic` is `trigger.WAIT_OR` or `trigger.WAIT_AND` and governs the whole list; the two cannot be mixed in one block (15-413). Three events in one block means a blender is unnecessary for an OR: "the analog trigger, or the front key, or the rear BNC" is one block.

The wait latches. An event that fires before execution reaches the block is remembered, and `clear` decides what happens to it:

`clear`	an event that already fired	measured
`trigger.CLEAR_NEVER` *(default)*	passes straight through	state `IDLE`, readings taken
`trigger.CLEAR_ENTER`	discarded; waits for a fresh one	state `WAITING`, 0 readings

The default is the dangerous one. A capture armed on the TRIGGER key with `CLEAR_NEVER` fires on a press made minutes earlier. `CLEAR_NEVER` is right when a notify is being used as a semaphore, where one raised before the partner reached its wait must still be honoured. The latch also clears at the start block and on exiting the wait, and a `BRANCH_ON_EVENT` consumes it, which is what makes a silence detector possible.

The default `logic` is `AND`, not `OR`. A wait built as `(event, clear)` with no logic argument disassembles as `LOGIC: AND`, while `LoopUntilEvent` emits `LOGIC: OR`. Harmless for one event, and a trap the moment a second is added.

7. Pre-trigger, and what the comparator needs

Digitize infinitely into a circular buffer, wait, then digitize a counted burst. The counted block stops the infinite one.

```
1) BUFFER_CLEAR      buf
2) MEASURE_DIGITIZE  buf, trigger.COUNT_INFINITE   -- fills continuously, wraps
3) WAIT              <event>, CLEAR_ENTER
4) MEASURE_DIGITIZE  buf, <post-trigger count>     -- stops the infinite block
```

At 1 kSa/s into a 5000-reading buffer with a timer firing the wait at 2 s and a post-trigger count of 500, `buf.n` read 503 at about 0.5 s, 1505 at about 1.5 s and 2501 final: 2001 pre-trigger readings plus 500 post. The buffer accumulates across the wait, so the window holds everything from `initiate()` onward and a capture armed this way cannot miss the start of the signal it waits for. Pre-trigger depth is bounded by buffer capacity rather than by a percentage.

The pre-roll is not optional. Drop the `COUNT_INFINITE` block and the program becomes

```
1) WAIT              EVENT_ANALOGTRIGGER, CLEAR_ENTER
2) MEASURE_DIGITIZE  buf, 400
```

which waits for ever. Three seconds on a live 1 kHz square wave, comparator armed dead centre of its 0-3 V swing, gave `STATE_WAITING` with zero readings, no event and no error. The same level, the same slope, the same signal and the same session fired in under 0.01 s once a `COUNT_INFINITE` block was put ahead of the wait.

`EVENT_ANALOGTRIGGER` is derived from the digitizer's sample stream, so the comparator evaluates only while a measure or digitize block is running. With the wait first, nothing is sampling, the comparator never sees the signal, and the wait cannot be satisfied. There is no diagnostic: the model sits, which looks

identical to a trigger that has not arrived yet. The broken program is two blocks instead of four and is on the fast path, so it looks better than the working one. It is on the fast path because it never does anything.

The restriction belongs to the comparator and not to first blocks. `EVENT_EXTERNAL` as block 1 works: a two-block program, `WAIT EVENT_EXTERNAL` then a counted digitize, completed inside the first second with its full 100 readings against a 100 Hz pulse into the rear TRIGGER IN, and sat the whole four-second window at zero readings with the marker switched off. `EVENT_DISPLAY` as block 1 works too. Four arms of `WAIT EVENT_DISPLAY` with a one-reading measure block behind it, polled from inside, the press injected through the instrument's own web front panel:

arm	result
no press at all	`n = 0`, `STATE_WAITING` at block 1
key enabled, pressed	`n = 1`, `STATE_IDLE`
key disabled via `display.setkeyenable`, pressed	`n = 0`, `STATE_WAITING`
key enabled again, pressed	`n = 1`, `STATE_IDLE`

A finished counted pre-roll satisfies a comparator wait only through the latch. 100 samples at 40 kS/s, finished, then `WAIT EVENT_ANALOGTRIGGER`: `IDLE` with 200 readings. Those 100 samples span 2.5 ms of a 1 kHz square, so an edge crossed the level while the block was running, and the default `CLEAR_NEVER` let the wait consume what that edge latched. The same program with `CLEAR_ENTER` sat the full window at exactly 100 readings, the pre-roll's own samples and not one more. The comparator does not evaluate once the measure block has stopped; what a finished pre-roll leaves behind is a pending event.

The wait also keeps the pre-roll running. An infinite digitize as the only block idles after about a dozen samples whatever `dmm.digitize.count` says, because the sequence ends and that stops the block. With the wait downstream, the same block filled a 20 000-reading buffer and sat in `WAITING`. The counted block stops the infinite one; the wait is what lets it run at all.

The comparator's level and slope are both honoured, measured against a 1 kHz 0-3 V square wave. Armed at 8 V, outside the signal's range, the model sat in `STATE_WAITING` indefinitely and never fired. Armed at mid-swing, reading either side of the trigger index, `SLOPE_RISING` gave -0.001 V before and 3.000 V after and `SLOPE_FALLING` gave 3.000 V before and -0.001 V after, each with a single partial-transition sample at the edge.

An explicit count on the block overrides `dmm.digitize.count`: count 7 set, block count 250, 250 readings taken. Counts of 8192, 32768 and 1 000 000 are all accepted.

The comparator fires about 7.3 microseconds after the crossing, fixed in time

Measured against a slow ramp, so the crossing can be located by interpolation rather than quantised to a sample. The latency is the distance between the interpolated crossing and the trigger point the pre-trigger reserve puts in the record:

`samplerate`	slew	latency, samples	latency, µs	µs less one sample
2 500	6.25 V/s	1.308	523.34	123.33
40 000	100 V/s	1.380	34.49	9.49
200 000	500 V/s	2.432	12.16	7.16
1 000 000	2 500 V/s	8.361	8.36	7.36

Between 200 kS/s and 1 MS/s the rate changes 5.0x, the latency in samples changes 3.44x, and the latency in microseconds changes 1.03x.

Two caveats. The trigger instant is attributed to the first post-trigger sample; attribute it to the last pre-trigger sample instead and every figure drops by exactly one sample period, which is the right-hand column. That is a definition rather than something the data can settle. And the 2 500 S/s row cannot resolve this at all: one sample is 400 µs there, so the half-sample ambiguity is 200 µs, thirty times the quantity being measured. Jitter is comparable to the latency, 6.2 to 9.3 µs at 1 MS/s and 3.6 to 9.9 µs at 200 kS/s, so the firing point is not deterministic to the sample.

`buf.n` is not a clock above 100 kS/s, but the sample rate is real. The ramp slope measures the true sample interval directly and recovers 1 000 673 to 1 000 996 S/s when 1 MS/s is asked, and 200 127 to 200 193 when 200 kS/s is asked. What lags behind the acquisition is `buf.n`, which is what the 2872/2873 "Begin/End reading backlog processing" pair reports.

8. Reading an armed record: the reserve

A window that holds everything from `initiate()` onward is also a hazard. **On an arm from silence the reserve is not a quiet stretch of the signal: it is a measurement of a switched-off wire, taken before the device under test transmitted at all.** Any reader that infers a property of the signal from the record's longest or leading stretch -- a polarity, an idle level, a baseline, a noise floor -- will take it from the reserve and be self-consistently wrong. The inference succeeds, reports a plausible quality figure, and is wrong in every sample it touches. Its length disarms the usual protection, which is a plausibility test asking whether the stretch is long enough to be a real idle: a reserve of a thousand samples clears any such threshold several times over, so the capture that most needs the sanity check is the one certain to pass it.

It is a percentage of capacity, which is what stops the permanent record from being used here. `sdec.acq_bufcap` allocates the recording buffer once at 6 000 000 readings so an eliding capture cannot outrun it, and the shipped position of 5 against that capacity is **300 000 readings** rather than 1055 -- a record fifteen times the depth, which the frame path copies whole into a Lua table. Loading 0 instead costs the head of the record: measured offline as three wrong bytes from offset 1, with no leading idle left for the level probe. So only a program that loads no position may take the permanent capacity, which is the elision program of section 10 and nothing else; `acq_pospct()` scales the figure for the one path that can reach `LoopUntilEvent` with that buffer underneath it. `docs/MEASUREMENTS.md`, 'The permanent record, and why only a recording may have it'.

Whether anything real out-measures the reserve is decided by the sample rate. The reserve is a fixed sample count, a percentage of buffer capacity, so 1055 samples at 5 % of a 21 100-reading buffer, while a gap in the signal is a fixed time. Armed on a silent line with the source switched on at +3 s, six captures a condition:

condition	leading run	next run	the next run in time
busy line, never armed	50	344	--
armed, 200 kS/s	1052	1210	6.05 ms
armed, 80 kS/s	1053	500	6.25 ms

The second run is the same gap in both armed rows while the leading run is the reserve at its fixed 1055 either way, so the reserve dominates at the lower rate and is overtaken at the higher one with nothing else changed. A defect of this kind therefore reproduces at one sample rate, vanishes at another, and looks

intermittent until the arithmetic is written down. The crossover is reserve divided by gap: 1055 samples against this 6.05 to 6.25 ms gap puts it at 169 to 174 kS/s.

Both polarities are affected, in opposite directions. On a single-supply line the dead wire sits at ground, which is also the signal's low level, so the reserve reads as a long low. On a ground-straddling line the threshold sits near 0 V and the dead wire sits all but on it, so which side the reserve falls on is decided by millivolts: at a -0.011 V threshold the dead line landed on the high side by 11 mV.

The remedy is to tell the reader that the record opens before its own trigger, with the flag conditional on the reserve existing: at a pre-trigger of 0 the record opens on the edge and its leading stretch really is the signal, so a flag set unconditionally is a constant and protects nothing. Exclude the leading run from any inference about the signal, publish it as its own quantity, and let the interior of the record decide. Where the window holds nothing but the reserve, the plausibility test fires honestly and the signalling standard's prior is used instead. With the leading run excluded, every armed condition above recovered to the same success rate as the never-armed control, and on a ground-straddling line 36 of 36 captures across three bands and two configurations read the polarity correctly.

Assert the premise as well as the outcome. A test that checks only the end result is vacuous when there was no reserve to begin with. Assert that the arm fired and that the leading run exceeded a few hundred samples: with the remedy disabled, the premise check fails in every configuration while the end-to-end check fails in only one of three, a downstream contest having rescued the other two. The premise check is the reliable detector; the outcome check is what says the defect reached a user.

9. Loops

Two mechanisms, both needed by any program that records more than one burst.

Detecting silence: `BRANCH_ON_EVENT` spends its event

There is no trigger event for a line going quiet. The comparator fires on a crossing and silence raises nothing, so no `WAIT` can be armed on it. The absence of a crossing over a window is still expressible, because `BRANCH_ON_EVENT` tests an event rather than blocking on it:

```
k)   DELAY_CONSTANT      0.1
k+1) BRANCH_ON_EVENT     <event>, k      a crossing arrived: go round
k+2) ...                                 fell through: quiet for 100 ms
```

The branch consumes the event, and the detector re-arms once per event. Three stimuli through one construct, with a `BRANCH_COUNTER` capped beyond reach at each landing site serving as a program counter:

event	stimulus	laps	branches	reading
`EVENT_NOTIFY1`	one notify block	2	1	consumed; fell through on re-entry
`EVENT_TIMER1`	0.5 s period	100	20	one branch per expiry, not per lap
`EVENT_ANALOGTRIGGER`	2 Hz square	100	20	one branch per crossing
`EVENT_ANALOGTRIGGER`	1 kHz square	100	100	every window busy; the loop never exits
`EVENT_ANALOGTRIGGER`	DC, no crossing	1	0	exits one window after silence

The notify row alone would not settle it: a detector consumed once and never re-armable predicts that same single branch and would be as useless. The periodic rows show the re-arm, and their counts are predictions rather than inequalities, 100 laps of 0.1 s against a 0.5 s period being 20 events. The manual leans the same way without saying it, "after the trigger event occurs, the next time the trigger model reaches the branch-on-event block, it goes to the branching block" (15-397), where "the next time" says nothing about the time after that. `BRANCH_ON_EVENT` takes no `clear` argument, so unlike `BLOCK_WAIT` the policy is not selectable.

The comparator keeps posting while the loop runs. Since `EVENT_ANALOGTRIGGER` comes from the digitizer's sample stream, it survives a loop made of delay and branch blocks only while a `COUNT_INFINITE` measure block is live. The comparator rows above were taken with that shape, and the DC row is their control: it exited after 1001 samples at 10 kS/s, 100.1 ms, one window and no more.

Two limits. The construct's resolution is its `DELAY_CONSTANT`, so a 0.1 s window cannot tell 0.1 s of quiet from 0.19 s. And the window is a minimum observable gap rather than a latency: a burst is detected as over no sooner than one window after its last edge.

A re-entered measure block appends, and the timestamps carry the splice

Put a counter behind the four-block pre-trigger shape and it records many bursts into one buffer while the silence between them goes elsewhere. The figures below were measured against this program, which is a bounded probe rather than the one the application ships -- "The six blocks" has that:

```
1) BUFFER_CLEAR      rec                       once, outside the loop
2) MEASURE_DIGITIZE  scr, COUNT_INFINITE       the gap's samples, discarded
3) WAIT              EVENT_ANALOGTRIGGER, CLEAR_ENTER
4) MEASURE_DIGITIZE  rec, 500                  the burst
5) BRANCH_COUNTER    4, 2
6) DELAY_CONSTANT    0
```

`rec` accumulates across passes. Four passes of 500 readings gave `rec.n` = 2000, not 500, so block 4 appends rather than restarting its record. `startindex` stayed 1, so a record that never fills is not rotated. The silence went where it was aimed: `scr.n` = 13 231, which is 1.3 s discarded against 0.2 s kept at 10 kS/s.

The span-derived sample rate is then wrong by the elision ratio. `(n−1)/span` over `buf.relativetimestamps` is how `sdec.acq_measure_fs()` reads the rate back, and it is the number every bit time is scaled by. Elision removes the gap's samples and not the gap's time, so on the record above the span was 1.5737 s for 2000 readings: 1270 S/s implied against 10 000 commanded, low by 7.9x. Any decode driven off the whole-span average is scaled by that factor.

The splices are exactly recoverable, which makes the rate repairable. Every reading keeps its own timestamp, so a gap appears as a discontinuity between consecutive ones. On that record: 3 splices, at indices 501, 1001 and 1501, the burst boundaries exactly, with deltas of 0.4743, 0.4497 and 0.4500 s against a commanded 0.5 s generator period. All 1996 remaining deltas were 100 µs, giving 10 000 S/s inside the bursts and a ratio of 1.000 against the commanded rate. Segment the record at the discontinuities and measure fs within a segment: only the whole-span average is corrupt.

The burst block can be infinite too, and the two measure blocks stop each other. Replacing block 4's count with `COUNT_INFINITE` and ending the burst with the silence detector gave `rec.n` = 4004 as four bursts of 1001 readings, splices at 1002, 2003 and 3004, gaps of 0.3983, 0.4001 and 0.4000 s, and 0.4 s kept against 1.198 s elided, which sums to the run. Each burst is 100.1 ms because the detector's window

is 0.1 s and the next crossing on that stimulus is half a second away, so the half period less the burst closes the arithmetic. The application takes that shape and drops two things from it, the counter and the clear block; see "The six blocks".

The failure this rules out would have been quiet rather than loud. Had block 4 not been stopped, both blocks would have kept writing: `rec` would have grown straight through the gaps, held more readings rather than fewer, and shown no discontinuity. `rec.n` on its own is not the oracle for such a program; the splice positions are.

But the re-entry is not instantaneous, and nothing above could see that. Every figure in this section was taken at 10 kS/s, where a reading is 100 µs. The application's record puts the resume cost at **156 to 208 µs** -- one or two readings here, invisible against burst lengths that came back at a flat 1001 -- so "the splices are exactly recoverable" is true of the timestamps and not of the wire. What is missing is the leading edge of whatever the generator sent first after the gap.

It is bounded from the seam debris of a decoded record rather than from a timestamp, because the timestamps cannot show time that was never sampled. Each seam's debris string depends only on how far into the first byte the record opened, so every observed string bounds it and the tight cases intersect at 156-208 µs. Scaled against a bit time it decides whether the seam is free:

baud	bit time	resume, in bits	the first frame after a seam
300 - 2400	3.33 ms - 417 µs	0.05 - 0.50	exact; samples stay inside their cells
4800	208 µs	0.75 - 1.00	fails, and 4 to 9 payload bytes die while the framer resynchronises
19200	52 µs	3.0 - 4.0	the same, further in

Half a bit is the threshold because a UART samples each cell at its centre: a record that opens `s` bit times late has no edge at the true start, the framer anchors on the splice instead, and every sample is late by `s`. Under 0.5 that is inside the cell. At 0.5 the stop sample lands in the next start bit, the frame fails, and the search for the next anchor walks three to eight bytes -- lowercase text offers no run of mark long enough to anchor on sooner.

So elision is byte-exact only where `0.5 / baud` exceeds the resume cost, which is about 2400 Bd and nothing above it. 2400 is a near miss rather than a margin: 208 µs is 0.4992 of a bit there.

A scratch that wraps costs nothing. A real gap of 2 to 120 s at 100 kS/s laps any affordable scratch, so the same program was run again with the scratch cut to 2000 readings, 0.2 s against a 0.4 s gap so it laps twice per gap, and nothing else moved: an identical record at 4004 readings in the same four bursts at the same indices, the same event-log delta of 3 entries, and the model still reaching `IDLE`. `fillmode = 1` is what buys that, and it has to be set numerically.

The discard costs no events at all. A wrapping scratch discards continuously for the whole gap, the panel shows 4915 as a modal dialog whatever `localnode.showevents` says, and reading the log empties it, so a flood would bury real events. Over recordings that elided 13.2 s of silence at 100 kS/s, about 1.3 million discarded readings, `eventlog.getcount()` read 0, with no error count, no fill failures and no sticky error. A wrap under `fillmode = 1` is not a discard the firmware reports.

10. The application's recording program

Recording mode acquires a long record of a serial line and decodes it in windows. It loads one of three programs, chosen in `sdec.stream_arm()` in `tsp/serial_app.tsp`:

condition	program
the line was already transmitting, or the trigger is free-running	`SimpleLoop(nsmp, 0, buf)`
the line is silent and the recording must wait for a device	`LoopUntilEvent(ev, pretrig, CLEAR_ENTER, 0, buf)`
the same, and gap elision is on with the comparator as the source	a hand-built 7-block program

`sdec.arm_silent()` makes the first decision: a recording waits only where the level probe found no line at all. Every refusal below falls back to the row above it rather than failing the capture, and the fallback to `SimpleLoop` acquires the instant it is initiated.

The arm's event is usually a blender

`sdec.arm_source()` returns the event, whether the comparator is a source, and whether a blender was used. The base event is `EVENT_ANALOGTRIGGER`, or `EVENT_DISPLAY` for the front-panel mode, or `EVENT_EXTERNAL` where the operator anchored the capture to the rear BNC alone. Two things are then OR'd in through `trigger.blender[1]`, which makes the event `EVENT_BLENDER1`.

The rear BNC is OR'd in rather than selected instead, because "start on a start bit, or when the device asserts its GPIO, whichever comes first" is not expressible as a single choice.

The front TRIGGER key is OR'd in as a manual break-in. A wait cannot be interrupted from Lua, but as a stimulus of the trigger model the key is serviced by firmware, so it is the only escape from an arm that is not going to fire. Four terms all have to hold: an arm from silence, `sdec.armkey` set, the key not already the source, and not an anchored capture, where only the marker may place the window.

Whether the comparator is a source is decided before the blender overwrites the event. Asking afterwards answered no for rear BNC with edge triggering, so the mode, level and slope went unset. The blender is reset when it is not in use: the model waits on its own event either way, but the next thing to use a blender would otherwise inherit this application's stimuli and its overrun flag. Where the firmware has no blender, the arm degrades to the rear BNC alone or to the unblended event, and `sdec.lasterr` says which capability was lost -- appended rather than assigned, because one firmware shortfall has two consequences and both are worth saying.

The comparator's level comes from `sdec.arm_threshold()`, which arms `sdec.armlevel` volts inside the idle level rather than at a measured midpoint: on a line waiting for a device there is no midpoint to measure. On a line idling at ground the start bit can only go up, so the arm goes above ground; on a line idling away from ground it goes on the ground side, clamped at half the idle level, the deepest arm certainly still inside the swing. `sdec.arm_slope()` takes the matching slope. Both are shared with the single-capture path so that the two cannot arm on different edges of the same line.

The six blocks

`sdec.gap_model()` in `tsp/serial_core.tsp` builds this, from `Empty`:

```
1) MEASURE_DIGITIZE  scr, COUNT_INFINITE     the gap
2) WAIT              <ev>, CLEAR_ENTER       the burst starts
3) MEASURE_DIGITIZE  rec, COUNT_INFINITE     the burst
4) DELAY_CONSTANT    <win>                   the detector's window
5) BRANCH_ON_EVENT   EVENT_ANALOGTRIGGER, 4  a crossing: still talking, go round
6) BRANCH_ALWAYS     1                       fell through: quiet for a window, next
burst
```

Blocks 1 and 3 are two measure blocks exactly, and entering either one stops the other. That is what ends a burst without counting it, and it is the whole reason the burst block can be infinite.

Block 2 takes `ev`, which may be the blender, and `CLEAR_ENTER`, which is load-bearing rather than cautious. The default `CLEAR_NEVER` lets this wait consume a crossing latched during the burst it has recorded, so it falls straight through, the next pass starts at once, and no gap is ever skipped -- while the record still fills and every other number still looks right.

Block 5 takes `EVENT_ANALOGTRIGGER` alone, and the difference from block 2 is deliberate. With `sdec.armkey` set the arm's event is a blender carrying the TRIGGER key, and a press arriving through it would read as "the line is still talking" and hold the burst open. Block 2 keeps the blended event for exactly the reason the blend exists; block 5 has to be the crossing by itself.

Block 6 is `BRANCH_ALWAYS 1` and not a counter. The recording's length is decided by the poll loop rather than by the program, which is why the program never terminates on its own.

There is no `BUFFER_CLEAR` block, and the record is cleared from Lua instead. `sdec.stream_arm()` calls `buf.clear()` immediately before the single `initiate()` that starts all three programs. The behaviour is identical: the program this one replaced carried a clear as block 1 and branched back to block 2, so the clear ran once per `initiate()` and never again, which is what one Lua call before `initiate()` does. The reason to move it is that a `BLOCK_BUFFER_CLEAR` is the only fast-path disqualifier either canned template carries, and the Lua call costs nothing -- so there is no reason to spend the disqualifier. **That is not a speed argument.** Being off the fast path is worth nothing measurable, 1.2 µs of upper bound against about 8 µs of comparator latency, as "Being off the fast path costs nothing measurable" sets out. The change is free, not fast.

A failed clear refuses the arm rather than being noted, with `could not clear the record before arming` in `sdec.lasterr`. This program has nothing to fall back on: the recording would append to whatever the last one left behind, and every span the rate is read from would then be long by the stale part, in the one direction nothing downstream flags. The two canned templates clear their own record, so for them the Lua call is redundant and costs one call.

The store is read back rather than trusted, because a rejected block reads as a free one. Four distinct opcodes must be present, `MEASURE_DIGITIZE` must occur exactly twice, and `BUFFER_CLEAR` **must be absent**. The absence is checked rather than the position, because this program has no safe position for a clear: block 6 branches to block 1, so every block is inside the loop and a clear anywhere among them runs at least once per burst -- at block 4 or 5, once per detector window, those two being an inner cycle. The record would then hold only the last burst, while `rec.n`, the splice positions and the recovered rate all stayed plausible for a recording of one. A listing missing block 3 describes a program whose bursts go nowhere, and one measure block is the pre-roll on its own.

Why the burst ends on silence and not on a count

A fixed burst count cuts wherever it lands, and the frame straddling the cut decodes from spliced samples. The loss is one byte in n frames, so a one per cent loss needs a hundred frames a burst, by which point a short gap is shorter than one iteration. Ending on measured silence puts the seam between an idle tail and the next start bit, which is a waveform the framer already reads: 43 bytes either side of a splice decode as 43 frames, not 42.

How long the line must stay quiet

`sdec.gap_elide()` returns the enforced window, or nil and the reason it declined. The capture and the options form both read it through that one function, so a typed value cannot mean one thing to the field and another to the trigger model.

`sdec.gapelide` is a threshold and not an amount. Once the window has elapsed the recording discards until the line moves again, so what comes out is unbounded and `sdec.gapelide_max`, 2.0 s, caps the threshold alone. It defaults to 0, which is off, because an elided record means something different: its time axis is not contiguous and no burst carries a pre-trigger reserve.

The floor is `sdec.gapelide_frames` frame times, and `gapelide_frames` is 2 rather than 1 because the constraint is the comparator's inter-crossing interval and the comparator arms on one slope only. With idle high and a falling arm, the rise out of a space is not a crossing at all, so on back-to-back `0x00` the falling crossings are one per frame. The window therefore has to clear `framebits`, `1 + nbits + parbits + nstop`, rather than the ten bit times a single byte suggests. At the widest forced format `framebits` reaches 13, where a rule written as 10 is 30 % short. Exceeding the floor costs one frame at the seam and not the record, the framer resynchronising on the next start bit, so this is a proportionate floor rather than a refusal.

`sdec.gapelide_min`, 0.002 s, is a clock floor, and past 10 000 Bd at 8N1 it is the binding one: two frames there are under 2 ms. Where the floor the rate derives exceeds the ceiling the window is refused rather than clamped, because clamping lets the ceiling win and hands back a window under the floor, which is the mid-frame cut the floor exists to prevent.

Elision needs a locked rate, since without one there is no frame to measure the floor against. Every path that can elide has one already, `sdec.stream_samples()` being unable to size a recording without it, so the decline covers the remaining case instead of guessing: `gap elision off -- lock the baud rate first`.

The window is the one cost that scales. It is recorded as idle at the end of every burst, because the detector cannot know the line stopped until the window has elapsed, so each burst carries `win * fs` readings of tail: 0.1 s at 100 kS/s is 10 000 of them. Expressed in bit times that cost is constant, about 20 times the samples per bit, which is why the floor is derived from the bit time.

The scratch

`sdec.gap_scratch()` makes the bin the gap's samples go into, `max(2000, ceil(0.25 * win * fs))` readings at `fillmode = 1`, and reuses it across captures while its capacity is enough.

Small and fixed rather than scaled to the gap is the one sizing here that looks wrong and is not. 2000 readings behaved identically to 400 000 on the instrument, giving the same record at the same indices, the same event-log delta and still reaching `IDLE`, because `fillmode = 1` laps the ring silently. A two-minute gap needs no more scratch than a half-second one. The scratch is deleted before its handle is dropped,

since assigning nil over the handle strands the readings in the pool instead of freeing them, and the pool is shared with the record. `sdec.gap_release()` hands it back when an operator turns elision off again, a buffer nobody is using being capacity the next capture cannot have.

How a recording ends

`sdec.stream_acquire()` polls at `sdec.ck_poll_s`, 0.5 s, and sets `sdec.ck_endwhy` on the way out. Five exits, and which completion test applies depends on which program loaded.

`full` means the recording got what it asked for. For an armed recording without elision that is `sdec.trig_done()`, the model reaching `STATE_IDLE`. For an eliding recording the sample count is the only completion available, because the program is a `BRANCH_ALWAYS` loop that never reaches `STATE_IDLE`, so `trig_done()` is a constant false. Left on the armed branch the fill test is unreachable, and because the record is `fillmode = 1` it would not stop but wrap, overwriting its own earliest bursts until some other exit fired, then report a full record that had quietly lost its oldest traffic.

`quiet` is the idle watchdog, and on an eliding record its timer comes from the model's state rather than from the record. Parked in block 2's `WAIT` means a gap; cycling blocks 3 to 6 means a burst. A record-derived timer cannot work here: the record does not grow during a gap, so a freshness test is false, and when the record is read `sdec.idle_window()` caps at 4000 samples, 40 ms at 100 kS/s, which sees only the burst's own idle tail. The timer would start 40 ms into every tail, before the gap had begun, and no allowance against that can be right. The model's state tells a long gap from a dead line exactly, which no threshold on a record designed not to grow can.

`stopped` is the operator: the TRIGGER key through the cancel latch, or Mode. The latch is read before the arm expiry, because TRIGGER is blended into the arm and a press both fires it and sets the latch.

`noarm` means the arm did not fire within `sdec.arm_wait_s()`. The level the comparator was armed at is stashed at the expiry, the next window clearing it before the note is printed.

`readfail` is 20 consecutive failed window reads. A read that raised is ignorance rather than silence, so it must not advance the quiet timer, and a persistent run of them is different from a transient one.

Over all of these sits a wall-clock ceiling from `sdec.stream_limit()`, which is the only bound that always fires: a touch press is not dispatched while Lua runs, so every mode has to terminate by construction. For an eliding recording the ceiling is `sdec.strm_maxsec` rather than `sdec.stream_maxwait()`'s twice-the-fill-time, which assumes the record fills in real time. An elided one does not. The arm's own wait is added on top of the ceiling rather than taken out of it.

One field for both waits

Arm/Idle Wait answers one question asked twice: how long this device may say nothing, before it starts and between blocks once it has. 10 s by default, 2 s minimum, 120 s ceiling, read by `sdec.arm_wait_s()` for the arm and by `sdec.idle_exit()` for the quiet-line exit. `idle_exit()` floors it at `sdec.idle_exit_s`, which is 1 s and so never binds against a 2 s minimum, and passes a zero or nil floor straight through as the watchdog's off switch.

A fixed 1 s exit ended a recording inside the first pause of a startup dump, which stops for seconds at a time between blocks with the interesting part still to come. The observable in the tests is the poll count: 3 polls at a 2 s wait against 11 at 10 s.

Reading the record back

An elided record needs three things done differently, all keyed on `sdec.strm_elide` and `sdec.acq_elided`, which record what loaded rather than what was asked for. An elision that fell back to `LoopUntilEvent` has to be read with the whole-span formula or its rate comes out wrong in the one direction nothing would flag.

The rate is measured inside the bursts by `sdec.acq_seg_fs()`. The whole-span formula divides by a span that still covers the silence while n counts only traffic, so it reads low by the elision ratio: 40 467 S/s against a commanded 100 000, measured through the application. `acq_seg_fs()` samples 40 windows of 100 consecutive readings and averages the deltas, treating anything past `sdec.segfs_gapmult` intervals, 8, as a gap. The margin there is enormous by construction, the smallest gap elision can produce being one detector window and so at least 20 samples per bit of intervals, about 200 against 8, while inside a burst every delta is one interval to 0.3 %. Averaging rather than taking the shortest delta matters: a delta jitters 32 ns on a 10.5 µs interval, so the minimum reads 0.25 % high, the same order as the rate misreports the manual lists as a known failure. Averaging 3960 of them puts the jitter at 0.005 %. A full walk would be 3 413 334 proxy reads on the largest mode, 74 s at 21.73 µs each.

`leadmax` is set only on the armed path that is not eliding. An elided record has no reserve at all, every burst opening at its own crossing with the samples before it in the scratch. Left set, `pretrig` defaulting to 5 would make `leadmax` a fiction, `acq_pretrig` would come back true, and `sig_idle` would treat a genuine leading run as the wire before the device started.

`primeskip` is `ceil(2 * win * fs)` on an elided record. The silence the format search may have to cross is not at the front but at the end of every burst, and the tail is uniform on `[win, 2 * win)` because block 5 only evaluates at multiples of the delay and the last crossing fell somewhere in the previous one.

What the operator sees

While an arm waits, `sdec.armnote()` writes the right-hand cell: `waiting 10s for 1.00V -- TRIGGER=go`, with the tail present only where the key really was blended in. It is painted once, because a touch press only queues while Lua runs and a static panel otherwise reads as a hang. When the trigger fires the cell is given back, the run's last full refresh having already happened.

During a sliced decode the status row reads `decoding N bytes (window i/n) -- press Capture`, or `preparing: <phase>` while there is no byte count yet, since "decoding 0 bytes" reads as a decode that found none.

At the end, an eliding recording that filled its buffer reports what elision bought: `buffer full -- 45 bursts, elided 13.2 s of 14.6 s`. The burst count is the half that makes it self-diagnosing, since elision pays only while a burst outlasts its own detector window and kept-against-spanned alone cannot tell a record that saved a minute from one that is mostly window. The seams are counted rather than divided out, a gap being however long the device stayed quiet. A quiet exit reads `<n> s with no line activity`.

Where elision was asked for and did not run, `sdec.gap_why()` appends the reason, because every way it declines is invisible and none is the operator's mistake. Flow control is reported before the rate: `sdec.arm_silent()` refuses outright while `sdec.fc_out` is set, so elision and flow control are mutually exclusive, and reporting the rate first would put `gap elision off -- lock the baud rate first` on the glass, an instruction that cannot work. The other reasons are a free-running trigger, a line already transmitting when the recording started, and no comparator armed. The last of those is the one that bites on the bursty line this feature targets: `sdec.arm_silent()` needs the level probe to have found no line, so a probe that lands inside a burst arms nothing and the recording falls back to `SimpleLoop`.

Those figures come from a run end to end through the application: 13.2 s of silence discarded, 45 bursts spliced, the repaired rate exact, and 1.3 million discarded readings leaving a completely empty event log.

11. The canned templates and the fast path

`LoopUntilEvent(event, position, clear, delay, buffer)`:

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE INFINITE
4) WAIT <event> CLEAR:ENTER           5) MEASURE_DIGITIZE <count>  6) NOTIFY 2
```

`position` is a pre-trigger percentage, and the post-trigger count the template programs is `(100 - position)` per cent of the buffer's capacity. `dmm.digitize.count` has no part in it: capacity and position were varied independently across 15 combinations, each matching the rule exactly, and `dmm.digitize.count` set to 77 against 1000 changed nothing.

capacity	position 0	5	20	50
10 000	10 000	9 500	8 000	5 000
20 000	20 000	19 000	16 000	10 000
50 000	50 000	47 500	40 000	25 000

An armed capture never fills its buffer, and the shortfall is not a fault. At `position` 5 the template programs 95 % of capacity as the post-trigger count, and the readings above that are whatever accumulated between `initiate()` and the trigger arriving. On a continuously active line that is a handful of samples rather than the 5 % the position nominally reserves, so a completed capture lands a little over 95 % of capacity and never at 100 %: 19 011 of 20 000 at 9600 baud, measured. Only an arm from silence runs the pre-roll long enough to fill the reserve. Testing for a full buffer as the completion condition fails on every capture that worked.

`SimpleLoop(count, delay, buffer)` is a counted loop with no wait at all:

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE <buf>, 1
4) NOTIFY 2      5) BRANCH_COUNTER <count> -> 2
```

It acquires the instant it is initiated, a recording built on it cannot be armed, and a 32 768-sample recording executes 32 768 iterations of a three-block loop.

The low-latency flag

`getblocklist()` ends with `* This trigger model may not be set up for low latency. *` whenever the program is off the fast path. That makes the boundary observable, and what it enforces is a list of specific disqualifiers, only one of which is a size limit.

disqualifier	measured boundary
measure/digitize blocks	1 and 2 keep it; 3 loses it
total blocks	14 keeps it; 15 loses it
`BUFFER_CLEAR`	any, even one

disqualifier	measured boundary
`DELAY_CONSTANT`	0, 0.1 and 0.25 s keep it; 0.3 s loses it, so the boundary lies in 250-300 ms
`LOG_EVENT`	any, with a valid event number
two blocks waiting on the same event	any
a configuration-list block (`CONFIG_RECALL` / `NEXT` / `PREV`)	any, even one
a branch to a block that does not exist	any

Free: `NOP`, `NOTIFY`, `WAIT` including a three-event `WAIT_OR`, `BRANCH_COUNTER`, `BRANCH_ALWAYS` to a target that exists, `COUNT_INFINITE`, a second measure block writing to a different buffer, and any number of `DELAY_CONSTANT` blocks -- six were free, so the delay disqualifier is duration and not count.

The firmware states the measure-block limit itself. Building a third measure block posts

```
2753: Trigger model is not optimized for speed due to greater than 2 measure/digitize
blocks
```

2753 is a warning and not a refusal, and its message names whichever cause applied rather than always naming measure blocks. The application's own recording program has exactly two measure blocks and posts 2753 on every run, with a different tail:

```
2753: Trigger model is not optimized for speed due to multiple blocks waiting
```

Measured over 131 cells of 132. So the event marks "off the fast path" generally, and the sentence after it is the part that says why. Reading the bare code as a measure-block count is what this document and three places in the tree used to do.

Two waits on one event is a real disqualifier, and the row most easily mistaken for something else. Holding the program shape fixed and varying only the events:

```
one WAIT on EVENT_DISPLAY, then a measure           low latency
two WAITs, both on EVENT_DISPLAY                    warned
two WAITs, on EVENT_DISPLAY and EVENT_EXTERNAL      low latency
```

It holds at two different block counts, so the cost is the shared event and not the extra blocks, which is why a `WAIT / MEASURE / WAIT / MEASURE` program warns while `WAIT` itself is free.

`BRANCH_ALWAYS` costs nothing; a dangling target costs everything. A `BRANCH_ALWAYS` to a block that exists is free, backwards or forwards, while the same opcode pointing past the end of the program warns. The disqualifier is the unresolvable target rather than the jump.

Being off the fast path costs nothing measurable

The flag is an oracle with no units attached, so the penalty was measured directly against the comparator latency, on three program shapes:

shape	blocks	low-latency warning	latency at 1 MS/s
A, the canned `LoopUntilEvent`	6	present	8.23 ± 1.26 µs (n=20)

shape	blocks	low-latency warning	latency at 1 MS/s
B, hand-built 3-block equivalent	3	absent	8.05 ± 0.98 µs (n=20)
C, A with only the `BUFFER_CLEAR` removed	5	absent	8.00 ± 1.00 µs (n=18)

B - A = -0.34 ± 0.32 µs and C - A = -0.49 ± 0.34 µs, inverse-variance weighted over the two rates that resolve microseconds. Both are 1.1 to 1.4 sigma, and the 95 % upper bound on the benefit is 1.2 µs against a total comparator latency of about 8 µs. Fast-path status was read from the firmware's own warning string rather than inferred from the disqualifier table.

C is a result in its own right: `BUFFER_CLEAR` is the sole disqualifier in the canned template. Removing it alone clears the warning, so the zero `DELAY_CONSTANT` and the `NOTIFY` cost nothing.

Throughput is a null as well, and no sample is dropped at the trigger. The fitted per-sample step of a ramp is the dropped-sample detector: 0.002498 V against 0.002500 programmed for all three shapes at both rates, and all three deliver the full programmed 20 045 post-trigger readings at 1 MS/s. The handover from the infinite pre-trigger digitize to the counted post-trigger one loses nothing, which is what makes the pre-trigger reserve possible.

One difference could not be removed and is inert: A's wait reports `LOGIC: OR` while both hand-built waits report `AND`, because `setblock` cannot take a logic argument without a second event. With one event the two mean the same thing. So there is no latency and no throughput case for restructuring a four-block armed capture, and the arm timing agrees from the other side: moving the clear out of the model does not shorten the arm either, 0.1763 s against 0.1788 s.

What the manual says about performance

The manual never uses the words "low latency", which is zero occurrences in 55 488 lines. The firmware's vocabulary for its own fast path is undocumented, and the manual nowhere connects the flag to its performance advice. That advice is seven bullets at 9-50, headed "Improving the performance of a trigger model". Four are what the flag enforces; three are wrong as written.

9-50 says	measured
fewer than 15 blocks	agrees: 14 keeps the flag, 15 loses it
constant delays under 254 ms	agrees: 0.25 s keeps it, 0.3 s loses it
four or fewer measure/digitize blocks	stricter than documented: only 2 keep it
do not use multiple reading buffers	contradicted: a second buffer costs nothing
four or fewer delay blocks	not enforced: six were free
no two blocks waiting on the same event	agrees, and it is enforced rather than advice
limit configuration-list blocks	agrees absolutely: one block is enough to lose the flag

The consequence is the opposite of what it looks like. The two armed shapes most worth building are both on the fast path:

```
MEASURE_DIGITIZE INFINITE / WAIT / MEASURE_DIGITIZE n     low latency   (pre-trigger)
WAIT / MEASURE_DIGITIZE n                                 low latency   (armed
recording)
```

while `LoopUntilEvent` and `SimpleLoop` are not, and the only reason is the `BUFFER_CLEAR` block each template emits. Every scan is off it too; the 29-block example below exceeds the 14-block limit on its own.

The disqualifiers have a shape. `BUFFER_CLEAR` is a side-effecting operation, a long delay blocks, a dangling branch target cannot be resolved, and two blocks waiting on one event cannot be ordered without knowing which consumes it, while a plain unconditional jump, a counted branch and a wait are all free. That is consistent with a fast path that resolves a bounded program ahead of time and falls back to general interpretation when it cannot. **The mechanism is open:** it is stated nowhere in the manual and nothing here establishes it, so treat the disqualifier table as the specification and the shape as a mnemonic.

12. The firmware's own scan program

The instrument writes trigger programs for itself, and reading one is the cheapest way to learn the idioms. A 6-channel RTD scan -- `scan.add("1,6,2,3,4,5")`, `scancount` 10000, `scaninterval` 10 s, `scan.export(..., WRITE_AT_END)`, on a `2000,10-Chan Mux` -- generates 29 blocks in two sequences:

```
A  1) START
   2) CONFIG_RECALL  ChannelList INDEX 1      -- the INIT step: open all
   3) CHANNEL_ACTION
   4) WAIT  EVENT_TIMER4   CLEAR:NEVER        -- the interval gate
   5) NOTIFY 5                                -- restarts timer 4
   6) CONFIG_RECALL  ChannelList INDEX 3
   7) BRANCH_ONCE   -> 11                     -- cold start: no prior measurement to
wait for
   8) BRANCH_ALWAYS -> 10
   9) CONFIG_NEXT    ChannelList
  10) WAIT  EVENT_NOTIFY7  CLEAR:NEVER        -- "the measurement finished"
  11) CHANNEL_ACTION                          -- close the next channel
  12) NOTIFY 6                                -- "the channel is ready"
  13) BRANCH_COUNTER 6     -> 9               -- inner loop: 6 channels
  14) BRANCH_COUNTER 10000 -> 4               -- outer loop: 10000 sweeps
  15) END

B 16) START
  17) BUFFER_CLEAR   defbuffer1
  18) WAIT  EVENT_TIMER4   CLEAR:NEVER        -- the same interval gate
  19) CONFIG_RECALL  MeasureList INDEX 3
  20) BRANCH_ALWAYS -> 22
  21) CONFIG_NEXT    MeasureList
  22) WAIT  EVENT_NOTIFY6  LOGIC:AND          -- "the channel is ready"
  23) MEASURE_DIGITIZE  defbuffer1  COUNT:AUTO
  24) NOTIFY 7                                -- "the measurement finished"
  25) BRANCH_COUNTER 6     -> 21
  26) NOTIFY 8                                -- "the sweep finished"
  27) BRANCH_COUNTER 10000 -> 18
  28) WRITE_USB      defbuffer1               -- scan.export, WRITE_AT_END
  29) END
```

Five idioms, none of them in the reference manual.

A ping-pong rendezvous. A closes a relay and raises `NOTIFY 6`; B waits on it, measures, raises `NOTIFY 7`; A waits on that before advancing. Neither sequence can outrun the other, and `CLEAR:NEVER` is what makes a notify behave as a semaphore rather than a race.

A self-restarting interval timer. `trigger.timer[4]` has `delay = 10`, `count = 1` and `start.stimulus = trigger.EVENT_NOTIFY5`. Blocks 4 and 18 wait on `EVENT_TIMER4`; block 5 raises `NOTIFY 5`, which starts the timer. Each sweep re-arms the clock at its own start, giving a period measured sweep-start to sweep-start rather than 10 s of dead time added to a sweep of unknown length. The timer configuration lives outside the trigger model, in the setup script.

Loop peeling with `BRANCH_ONCE`. Block 7 skips the `WAIT NOTIFY7` exactly once, because on the first channel of the first sweep there is no earlier measurement to wait for. Every later pass falls through to block 8 and waits properly. One block replaces a prologue.

Configuration lists as the data plane. Per-channel settings are not expressed as branches. Two lists, `ScanAutoCreatedChannelList` and `ScanAutoCreatedMeasureList`, are stepped by `CONFIG_NEXT` inside each inner loop, and `MEASURE_DIGITIZE` uses `COUNT:AUTO` to take each channel's own count from the recalled configuration. The program is the control flow; the lists are the parameters.

No blenders. Both are untouched, `orenable` false and every stimulus `EVENT_NONE`. Keithley synchronises with notifies and timers alone.

Two cautions the same example demonstrates. `WRITE_AT_END` puts the USB write in block 28, so a power loss anywhere in the preceding 27.8 hours costs the entire record. And `scan.monitor.channel` set without `scan.monitor.mode` generates no limit or branch-on-limit blocks at all, leaving the monitor inert while the scan runs on its interval.

The first sweep stalls for one whole interval. A timer carrying a `start.stimulus` does not start when it is enabled: `timer[1]` with `delay = 1.0`, `start.stimulus = EVENT_NOTIFY5` and `enable = ON`, with a program waiting on `EVENT_TIMER1`, sat in `STATE_WAITING` with 0 readings three seconds later. Nothing raises `EVENT_TIMER4` until block 5 runs, and block 5 is downstream of the wait at block 4. `CLEAR:NEVER` cannot rescue it, a latch needing an expiry to latch and there having been none. So the scan takes `scaninterval` seconds to produce its first reading, which on a 27.8-hour run is invisible.

13. Where the vendor manuals are wrong

- **The notify count is given inconsistently, and the wrong value dominates.** Nineteen places in the event tables say "Notify trigger block N (1 to 3)", from 9-54 through 15-433, against six that say "1 to 8" (13-221, 13-239, 13-265, 15-383, 15-397, 15-413). Eight is right: `trigger.EVENT_NOTIFY8` resolves on the instrument, and the scan generator emits `NOTIFY 6`, `7` and `8`.
- **Out-of-band failure reporting is undocumented.** Nothing states that `setblock` and `initiate` report refusals as event-log entries while returning success, and no example checks for it.
- **A constant that does not exist.** The SCPI mapping table at 9-52 gives the paused state as `trigger.STATE_PAUSE`. That constant is nil on the instrument; the real one is `trigger.STATE_PAUSED`, as 15-414 has it, and the state it names is reachable.
- **A buffer constant the manual uses and the firmware does not have.** `buffer.FILL_CONTINUOUS` occurs six times in both reference revisions, including a worked example at 7-9 and a parameter table giving its numeric value as 1, and the constant does not exist on 1.7.17a. Assigning it raises. The attribute takes the number, so `fillmode = 1` reads back as 1 and is the only route to continuous fill.
- **A second absent constant, which fails as a trigger-model error.** `buffer.STYLE_COMPACT` is nil on 1.7.17a, so `buffer.make(n, buffer.STYLE_COMPACT)` hands nil to the style parameter and posts 1138, "Parameter 2, expected type 'rbuffer_style', but found type 'nil'". The cost is not one event: every `setblock` built on the nil result posts 2703, the program reaches `STATE_FAILED`, and `pcall` returns true throughout, so the symptom presents as a broken trigger model while the cause is a buffer

constant four events earlier in the log. `buffer.STYLE_STANDARD` resolves. No DMM6500 manual names `STYLE_COMPACT` at all; the DMM7510's does, which is how it reaches ported code.

- **Four undocumented block types.** `START`, `END`, `CHANNEL_ACTION` and `WRITE_USB` each occur zero times in both reference revisions and in the user manual. The first three exist as TSP constants and appear in no `setblock` usage list; `WRITE_USB` is emitted by `scan.export()`.
- **An exhausted branch counter reads 0, not one past its target.** Page 15-391 has it that "when the trigger model reaches the set counter value, branching stops and the counter value is one greater than the setting", so a counter set to 20 should read 21 afterwards. It reads 0. One model, one statement, two counters: the free-running one read 20 while the exhausted one read 0, and the same pair read 11 and 10 mid-run, so both were counting normally until one hit its target. End-of-run reset is ruled out separately, the free-running counter having been read after the model reached `IDLE` and still held its count. The firmware self-resets a counter on exhaustion, which is what a nested inner loop needs and what the documented value would break. A counter read back as a program counter must therefore be capped beyond reach, or it reports zero in exactly the case where it ran every lap.